

SUPERVISED MACHINE LEARNING
K-Nearest Neighbors Classification

Jean René RANDRIANJAKAMANANA

2024-12-02

Table de matières

IV) CLASSIFICATION	2
1) K-Nearest Kneighbors Classification	2

IV) CLASSIFICATION

1) K-Nearest Kneighbors Classification

Let use the datasets of **TITANIC'S WRECK**

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4 import pandas as pd
5
6 from sklearn.datasets import make_regression
7 from sklearn.linear_model import LinearRegression
8 from sklearn.metrics import mean_squared_error, r2_score
```

```
1 titanic_real = pd.read_csv("titanic.csv")
2 titanic_real.describe()
```

	Survived	Pclass	Age	Siblings/Spouses Aboard	Parents/Children Aboard	Fare
count	887.000000	887.000000	887.000000	887.000000	887.000000	887.000000
mean	0.385569	2.305524	29.471443	0.525366	0.383315	32.30542
std	0.487004	0.836662	14.121908	1.104669	0.807466	49.78204
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.250000	0.000000	0.000000	7.92500
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.45420
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.13750
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.32920

```
1 titanic = titanic_real[['Survived', 'Pclass', 'Sex', 'Age']]
2 titanic = titanic.dropna(axis= 0)
3 titanic['Sex'].replace(['male', 'female'], [0,1], inplace = True)
4 titanic.head()
```

	Survived	Pclass	Sex	Age
0	0	3	0	22.0
1	1	1	1	38.0
2	1	3	1	26.0
3	1	1	1	35.0
4	0	3	0	35.0

The main purpose is to create a model of classification which can determine if one passenger of Titanic survived or not. There are lots of classification models such: SGD Classifier, Naive Bayes, Linear SVC, but we try to learn the basic with **KNeighbors Classifier**

```
1 from sklearn.neighbors import KNeighborsClassifier
2
3 # KNeighbors Classifier
4 modele = KNeighborsClassifier(n_neighbors = 5)
5
6 y = titanic['Survived']
7 X = titanic.drop('Survived', axis = 1)
8
9 modele.fit(X, y)
10
11 print(f"Score (error only between prediction and true value y) = {modele.score(X,y)}")
12 y_pred = modele.predict(X)
13
14 R2 = r2_score(y, y_pred)
15 rmse = mean_squared_error(y, y_pred)
16
17 print(f"Score (error created by the entire datasets) = {R2}")
18
19 rmse
20
21 def survive(modele, Pclass = 3, Sex = 0, Age = 25):
22     X1 = np.array([Pclass, Sex, Age]).reshape(1,3)
23     X = pd.DataFrame(X1, columns = ["Pclass", "Sex", "Age"])
24     print(modele.predict(X))
25     print(modele.predict_proba(X))
26
27 survive(modele)
```

Score (error only between prediction and true value y) = 0.8534385569334837

Score (error created by the entire datasets) = 0.38135093084392935

[0]

[[0.8 0.2]]